

K-mer graph practical

Context

This practical explores genome assembly, assembly evaluation and assembly graphs. To keep the practical short while making the comparisons interesting, we use two small synthetic genomes of approximately **500 kb**, with repeats and sequence differences.

You have **Illumina-like paired-end reads**, **Oxford Nanopore-like reads** and **PacBio HiFi-like reads** for both genomes, and a **reference genome for sample A**.

You are free to choose your datasets, assemblers and parameters. The goal is to understand the choices you make and their effects on the assembly.

Use the **cheatsheet cautiously**: consult it to check your commands or when you feel blocked. The goal is to learn, not simply to enter command lines.

Main steps

1. Data examination
2. Assembly
3. Evaluation
4. Explore assembly graphs and compare the two genomes.

Analyze your data

Use **SeqKit** to investigate the properties of the datasets. For each dataset, consider the following questions:

- How many bases and reads are there?
- What are the mean read length and the N50?
- What are the lengths of the longest and shortest reads?
- Can you estimate the depth, given that the genome size is approximately 500 kb?

Your first assembly

The goal of this section is to perform an initial assembly using one of the three types of reads provided.

You may choose any of the available assemblers below. The purpose of the practical is to encourage you to take the initiative and make a choice. For this first attempt, if the assembler requires parameters, try to estimate reasonable values without overthinking. At this stage, the quality of the assembly is not critical.

Available assemblers

Short-read assemblers

Input: Illumina-like paired-end reads.

- **SPAdes** assembles short reads using de Bruijn graphs and can use several k-mer sizes. You can choose a single k-mer size and skip read correction for a faster comparison on these synthetic reads.
- **MEGAHIT** is a fast short-read assembler that also uses several k-mer sizes. It provides an alternative to compare with SPAdes.
- **GGCAT** constructs compacted de Bruijn graphs and outputs unitigs and links. Unlike assemblers that simplify the graph, it does not remove tips or collapse bubbles. Compare the graph and unitigs at different k-mer sizes.

Long-read assemblers

Input: Nanopore-like reads and/or PacBio HiFi-like reads, according to the tool.

- **Flye** builds a repeat graph from long reads. You can use it separately with the Nanopore-like or HiFi-like reads.
- **hifiasm** assembles PacBio HiFi reads. It can also use ultralong Nanopore reads alongside HiFi reads to help resolve repeats.
- **Verkko** combines accurate long reads and ultralong Nanopore reads in a hybrid assembly.

Exploration

Paired-end versus unpaired reads

Short-read assemblers can use pairing information to improve an assembly. Try ignoring pairing information while keeping the same input reads, and compare the results. Does pairing affect contigs, scaffolds, or both?

K-mer sizes

Assemblers relying on k-mers can produce different results depending on the value of k . Experiment with k-mer sizes or examine intermediate assemblies. Does a larger k resolve every repeat? Would you choose the same value for both samples?

Read length and hybrid assembly

Compare assemblies from the different technologies. Which repeats can long reads resolve that short reads cannot? Can read accuracy compensate for reads that do not span a repeat?

You can also combine the HiFi-like and Nanopore-like reads with hifiasm or Verkko. Compare the hybrid result with assemblies from each technology alone.

Assembly evaluation

Once you obtain an assembly, assess the quality of its contigs. You can use reference-free evaluation, reference-based evaluation and graph exploration.

Reference-free evaluation

Merqury evaluates an assembly by comparing its k-mers with those found in accurate reads from the same sample.

First, use **Meryl** to count k-mers from the Illumina-like reads. Then use **Merqury** with the resulting database and your assembly.

Examine the estimated consensus quality, k-mer completeness and copy-number spectra. Frequent read k-mers should generally be represented in the assembly; rare k-mers can reflect sequencing errors.

- Can a fragmented assembly still have high consensus quality?
- What would missing k-mers or excessive assembly copies look like?
- Can k-mer evidence detect every wrong join or collapsed repeat?

Use A reads to evaluate A assemblies and B reads to evaluate B assemblies.

Reference-based evaluation

QUAST compares an assembly with a reference genome using sequence alignments. The supplied reference corresponds to **sample A**.

Compare several A assemblies in the same analysis. Examine the contig alignment viewer as well as the summary statistics.

- How many misassemblies were made by the assembler?
- What percentage of the reference is covered by the contigs?
- How contiguous is the assembly? Compare N50 and NGA50.
- How close are the assembled sequences to the reference?
- Does the assembly with the largest N50 also recover the most reference sequence?

The A reference is not the true genome of B: differences between B and A can include biological differences as well as assembly errors.

Visualizing de Bruijn graphs

Vizitig can visualize the de Bruijn graph from any FASTA file, containing reads or assemblies. Build the graph with **GGCAT** at your chosen k-mer size.

For a read graph, examine how connectivity changes with the k-mer size. For an assembly graph rebuilt with GGCAT, remember that the graph represents sequence sharing at the chosen k ; it does not preserve every decision in the assembler's original graph.

- Is the graph well connected?
- Does it have many branching nodes?
- Which branches could correspond to unresolved repeats?
- How does changing k affect the graph?
- What information would help resolve an ambiguous path?

Two assemblies, one graph

Build the union of the two assembly sequences with **GGCAT**, then explore it in **Vizitig**. Colour the graph according to sequence membership in A and B to distinguish shared sequence from sample-specific sequence.

Look for small bubbles and larger alternative paths. Inspect their sequences, lengths, connections and orientations.

- Can you find a candidate single-nucleotide difference? Compare the sequences of the two paths with Vizitig's aligner.
- Can you find a candidate insertion, deletion or inversion?
- Do the alternative paths connect the same shared flanks?
- Could a sample-specific region instead reflect missing coverage or an assembly error?
- Does the interpretation remain consistent in an assembly from another technology or a graph built with another k-mer size?

Compare and repeat

Try different datasets, combinations of datasets, assemblers and parameters. Evaluate the resulting assemblies with **Mercury**, **QUAST** and **Vizitig**.

Once you have compared your assemblies, consider these questions:

1. What are the drawbacks of short-read assembly?
2. What are the drawbacks of long-read assembly?
3. What are the drawbacks of HiFi-read assembly?

How do pairing, read length, accuracy and depth affect your results? What do the graphs explain that assembly statistics alone do not?

Tool installation

The course Docker image already contains these tools. The instructions below are for a separate installation on Linux (x86-64), using the versions tested for this practical.

Installation environment

Install Miniforge if Conda is not already available. Create and activate an environment once:

```
conda create -n kmer-graph -c conda-forge python=3.12 pip
conda activate kmer-graph
```

Run the installation commands below in this environment. In a new terminal, activate it again with `conda activate kmer-graph`.

Short-read assemblers

SPAdes

```
conda install -c conda-forge -c bioconda spades=4.3.0
spades.py --version
```

MEGAHIT

```
conda install -c conda-forge -c bioconda megahit=1.2.9
megahit --version
```

GGCAT

```
conda install -c conda-forge -c bioconda ggcath=2.2.0
ggcath --version
```

Long-read assemblers

Flye

```
conda install -c conda-forge -c bioconda flye=2.9.6  
flye --version
```

hifiasm

```
conda install -c conda-forge -c bioconda hifiasm=0.25.0  
hifiasm --version
```

Verkko

Install bc, which Verkko uses for calculations, alongside the assembler.

```
conda install -c conda-forge -c bioconda verkko=2.3.2 bc=1.07.1  
verkko --version
```

Read and assembly statistics

SeqKit

```
conda install -c conda-forge -c bioconda seqkit=2.14.0  
seqkit version
```

QUAST

```
conda install -c conda-forge -c bioconda quast=5.3.0  
quast.py --version
```

Reference-free assembly evaluation

Meryl

```
conda install -c conda-forge -c bioconda meryl=1.4.2
meryl --version
```

Mercury

The Conda package also installs the R plotting dependencies, BEDTools and SAMtools. MERQUERY tells its scripts where to find their supporting files; set it again after activating the environment in a new terminal.

```
conda install -c conda-forge -c bioconda mercury=1.4.1 bc=1.07.1
export MERQUERY="$CONDA_PREFIX/share/mercury"
Rscript "$MERQUERY/plot/plot_spectra_cn.R" --help
```

Mercury documentation.

Graph visualization

Vizitig

Install Vizitig after the Conda packages above. Pin NetworkX to 2.8 for compatibility with the tested Vizitig and NetworkDisk versions.

```
python -m pip install vizitig==1.2 vizibridge==0.6.2
python -m pip install networkdisk==1.1.9 networkx==2.8
vizitig --help
```

The help command checks that the program starts; it does not open a graph. See the cheatsheet for graph construction and visualization commands.

Vizitig documentation.